HTA Firmware Tracker

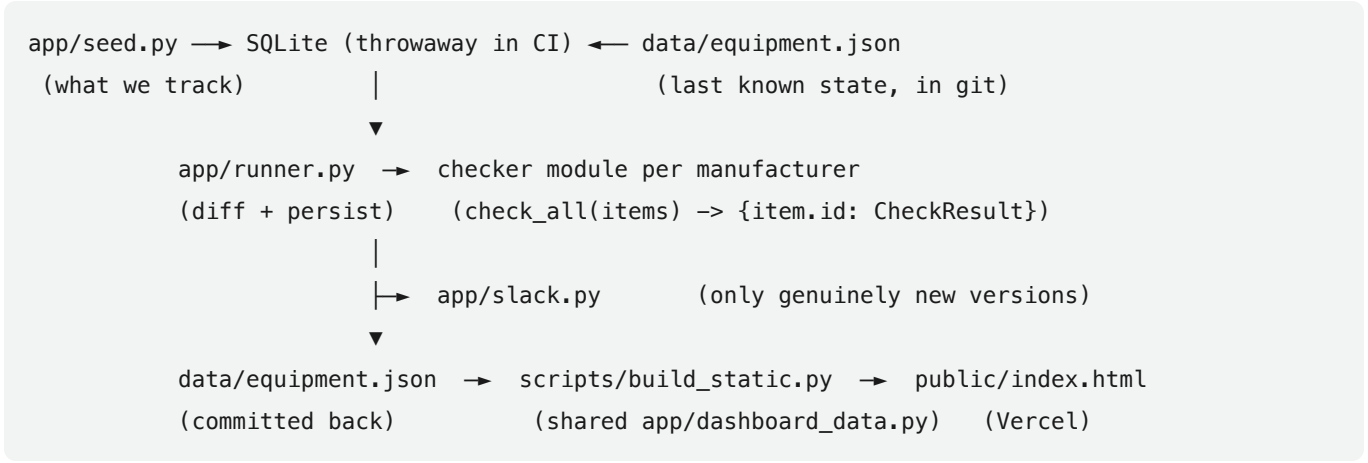
A web dashboard that tracks the latest firmware/software versions for HTA's audio gear, so nobody has to manually check manufacturer sites.

What it does

- Tracks 48 pieces of equipment across DiGiCo, Yamaha, Solid State Logic, Allen & Heath, Shure, d&b audiotechnik, and Dante/Audinate.
- **All 48 items are checked automatically** — most by scraping a static page, several manufacturers via a real headless browser where the site needs JavaScript or blocks plain requests (see How each manufacturer is checked).
- Re-checks everything once a day (via a scheduled GitHub Actions workflow - see Deployment - or via a background job if you run it locally) and highlights anything whose version changed since the last check.
- A stats panel at the top (Tracked / Updates / Errors / Last checked, the last shown as a relative time like "2h ago"), with a search box and Up to date / Update available / Check failed filters. Clicking the Updates card jumps straight to the flagged rows.
- Dashboard grouped by manufacturer, with status badges (Up to date / Update available / Check failed). A row with a new version gets a soft amber highlight in place (it isn't reordered) that clears itself automatically 1 month after the change was first detected — no one needs to click anything to dismiss it. Installable as a PWA on a phone (Add to Home Screen) for a full-screen, app-like view.
- Items that are genuinely the same underlying platform tracked from one source — e.g. DiGiCo's seven Quantum consoles — collapse into a single family row ("Quantum series", under DiGiCo's own section heading) instead of listing near-duplicate rows that always move together. See Firmware families for how that grouping is decided and kept in sync with Slack notifications.

Architecture at a glance

One flow, run in two places (a daily GitHub Actions job, and locally on demand):



- `seed.py` **is the list of what's tracked**; each row names a `checker_key` (e.g. `digico:quantum`), whose prefix `registry.py` routes to a manufacturer module. A checker gets all of its items at once and returns one `CheckResult` per item - it fetches each *source* once, not once per model.
- **State lives in git, not a database server.** CI starts from nothing each run: it re-seeds a throwaway SQLite DB, overlays the last known versions/timestamps from `data/equipment.json` (matched on manufacturer + model), checks, and writes the result back. So history persists as commits, and "what changed" is a before/after diff of two JSON snapshots (`app/changes.py`) rather than anything stored.
- **The deployed site is static HTML.** `build_static.py` renders the same Jinja template and calls the same `build_dashboard_context()` as the live FastAPI app, so the two can't drift on grouping/sorting/stats. There's no server in production - and so nothing that writes: flags clear themselves after a month instead of needing an "acknowledge".
- **Failures degrade, they don't cascade.** A checker that raises marks only its own items "Check failed"; a Slack outage or missing webhook never blocks the daily commit; a release-date lookup failing doesn't fail the version check it rode along with.

Quick start

```
cd "hta-firmware-tracker"
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
playwright install chromium
uvicorn app.main:app --reload
```

Open http://127.0.0.1:8000 — the database (SQLite, at `data/tracker.db`) is created and seeded automatically on first run, and a check kicks off ~5 seconds after startup. (The `playwright install chromium` step downloads a ~90MB headless browser used only by the Allen & Heath / Shure / d&b / Dante checkers — see below.)

How each manufacturer is checked

Manufacturer	Method	Notes
DiGiCo	✅ Scraped	`support.digico.biz`'s Zendesk help-center JSON API. Console software (Quantum, SD) is read per model from the article's download buttons, not its title - see below
Yamaha	✅ Scraped	Each product's Downloads page on `usa.yamaha.com` (a `#firmware-table` with a genuine "Last Update" date per file, not just a version)
Solid State Logic	✅ Scraped	`support.solidstatelogic.com`'s Zendesk API too; tracked via SOLSA (versions 1:1 with SSL Live console software) and the Network I/O firmware bundle
Allen & Heath	✅ Scraped (headless)	Version: `www.allen-heath.com` returns HTTP 403 to plain requests (bot protection) — works fine in a real headless browser. Release date: a separate evergreen article on `support.allen-heath.com` (a Zendesk help centre, read via its public JSON API — even a real browser can't get past that subdomain's own Cloudflare challenge page), month/year only, no day
Dante/Audinate	✅ Scraped (headless)	The version table is inside a click-to-expand accordion, so it's not in the page until JS runs. Dante Controller is published as separate Windows / macOS Apple Silicon / macOS Intel builds with their own versions, so all three are tracked (see below)
d&b audiotechnik	✅ Scraped (headless)	Their Download Center is a JS-driven search box behind a cookie-consent overlay; D40/D90 share one firmware release, DN1 Switch has its own
Shure	✅ Scraped (headless)	Firmware versions come from `shure.com`'s searchable software/firmware archive listing

Every item is checked automatically. A piece of gear only gets added to the list once a checker can read its version.

Per-platform versions (Dante Controller). Its row stays compact - one status badge and the newest version, with a "3 platforms" note - and expands (chevron on desktop, tap on phone) to list each platform's own version. Each platform tracks its own previous version and change date, and the row's badge is "Update available" whenever *any* platform changed within the 1-month highlight window (the expanded list tags which one). Slack alerts name the platform(s), e.g. "Dante Controller (Windows) → 4.18.1.3", and an update to just one platform is caught even if it doesn't change the headline version. Each platform also has its own real release date - Audinate publishes a dedicated "Release Notes" page per platform build, linked right from the same accordion the version comes from - so the expanded list shows each platform's own date, and the row's own Release date column shows whichever platform's date matches the headline version.

Article-based sources (DiGiCo, SSL). These manufacturers announce firmware in a help-centre *article* rather than on a dedicated firmware page, so a new release normally shows up as a new article/title/version and is handled like any other update. DiGiCo's console software (Quantum, SD) is the exception, in two ways.

First, it keeps one article per major release and edits it in place as point releases ship - v2242 in June, then v2242.4 in October for only the Q112/Q225/Q225DS - without changing the title or creation date, which are

all a title-based check can see. That's how the tracker once showed Quantum at "V22" while v2242.4 was out. So each tracked model is matched to its own download link in the article body: the version comes from the file name (`Quantum1_V2242-4x_Update_Package.zip` -> V2242.4) and the release month from the upload path (`/2026/10/` -> Oct 2026). If a model isn't listed it shows "Check failed" rather than quietly falling back to the title. The Q225DS has its own package, matched separately (DiGiCo's "Dual Screen" is treated as "DS"). Not tracked: the Quantum 852.

Second, DiGiCo reorganises these articles without notice - the title has gone "Quantum Console Software V2126" -> "V22 Quantum Console Software" -> "Version 22 Quantum Software", and the links from buttons to a dropdown (with "Quantum 326 & 338" becoming "326 / 338"). On 2026-11-06 a rename stopped the title match and the checker silently read an *older* article, recording V1619 and V2126 as new "updates" and failing three models. So the article is now chosen by its contents - the newest one with console `Update_Package` files for the family, whatever it's called - and a tripwire fails the check loudly if a newer article's title looks like this family's console software but has no links we can read, rather than reading an older release. A failed check also no longer cancels an in-window "update available" flag when it recovers. SSL picks its article by title the same way the old DiGiCo check did, so it has the same exposure to a rename. The DMI-Dante64@96 card is tracked as two rows (Zynq HC and the older Summit HC), since they're separate hardware variants with their own firmware lines - one shared row used to flip between whichever line published most recently.

Every manufacturer publishes a real release date now. The "Detected on" fallback (the day the tracker itself first caught a version change, `last_changed_at`, not a manufacturer-published date) is still there for whenever a future manufacturer's checker has nothing to put in release_date - it's determined per manufacturer from the data (any item with a real release date), not a hardcoded list, so nothing needs to change here if that happens again.

How the headless-browser checkers work

`app/checkers/browser_base.py` holds a small Playwright helper (`browser_page()` launches a headless Chromium tab, `dismiss_cookie_banner()` clicks past consent overlays). `allenheath.py`, `dante.py`, `dbaudio.py`, and `shure.py` use it to load a page, interact with it if needed (click an accordion, type into a search box), and pull the version out of the rendered text. They're slower than the plain-HTTP checkers (each spins up a real browser) but run fine as part of the once-a-day background check. To add another one, copy the shape of `dante.py`, then register it in `app/checkers/registry.py` and point the relevant `app/seed.py` rows at it with `"scrape"` + a `checker_key`.

Firmware families

Some rows track the same underlying platform from one firmware source - DiGiCo's seven Quantum consoles, Allen & Heath's four dLive surfaces/ MixRacks, Yamaha's two Rio-D2 I/O racks, Yamaha's eight Rivage PM components, d&b's three D-series amps - and normally report the identical version because they're assigned the same `CheckResult` from one `check_all()` call. Those collapse into a single dashboard row (e.g. "Quantum series", under DiGiCo's own section heading) instead of six near-identical ones. On desktop/tablet every field - status, version, release date, previous version, last checked, source -

still shows directly in that one row, since it's identical for every member; only the individual model names (e.g. Quantum 112/225/225DS/326/338/5/7) sit behind the chevron, as pill tags. On phone, where the whole row's detail is already a tap away, the model pills join the rest of that existing reveal instead of getting their own control. A family whose collective name already spells out every member - d&b's amp group is named "D25/D40/D90", not a generic "series" label, because unlike the others it has no shorter collective name that wouldn't also fit some other d&b product - skips the chevron/expand entirely on both desktop and phone, in `FAMILY_NO_EXPAND` (`app/families.py`): there's nothing left for it to reveal.

Members that share a version stay together; ones that don't, don't. Sharing a source doesn't guarantee sharing a version - DiGiCo's v2242.4 shipped for three Quantum models while the rest stayed on v2242. So `_consolidate_families` groups a family's members by their *current version*: each group of two or more is one collapsed row, and a lone member on its own version is a plain row. In that Quantum case you get two collapsed rows - "Quantum series (112, 225, 225DS)" on V2242.4 and "Quantum series (5, 7, 326, 338)" on V2242 - each with its own pills, version, release date and status. The models are in the name (the shared leading words dropped) because two rows both called "Quantum series" would look like a duplicate; for a family whose name already lists its members it's just the subset ("D40/D90"). It's all recomputed from the current versions on every build, so it's back to one row the moment everyone matches. A member with no version yet (newly added, not yet checked) rides along with the family when there's only one version, and stands alone when there are several. A lone member split off a family with its own subsection (Rivage PM) stays in that subsection. Slack follows the same idea: one update message per distinct new version, naming the models when only part of a family moved ("DiGiCo Quantum series (Quantum 112, Quantum 225) -> V2242.4"). For this to matter a checker has to be able to tell members apart, which only some sources allow: DiGiCo (per-model download buttons) and the Yamaha Rio pair (two separate rows on Yamaha's page, each read by model name) do. The rest publish one version for the whole family by their own documentation - Yamaha's compatibility chart for Rivage PM, one combined release note for d&b's amps, one firmware download for dLive, one SOLSA article / firmware bundle for SSL - so there's nothing to split; Shure isn't a family at all (one row, one check per product).

This is deliberately **not** based on matching version-number strings - two unrelated products could coincidentally share a version number (Shure's AD610, SBC240, and ADXR are different product types and always stay as separate rows even if that happens). The real signal is `checker_key` (assigned per actual firmware source in `app/seed.py`): items sharing one were checked together and are guaranteed to move together. `app/families.py` holds the one `FAMILY_NAMES` mapping (checker_key → friendly collective name) that both `app/dashboard_data.py` (row consolidation) and `app/slack.py` (notification grouping, see Slack notifications) import, so the two can never disagree about what counts as "the same family." Those names are deliberately brand-free ("Quantum series", not "DiGiCo Quantum series") - on the dashboard the row already sits under its manufacturer's own section heading, so repeating the brand on every row under it would be redundant. Slack messages have no such heading, so `app/slack.py` prepends the manufacturer back on when it builds the notification text (correcting Yamaha's all-caps `YAMAHA` field to "Yamaha" through `MANUFACTURER_LABELS` along the way) - a Slack message still reads "DiGiCo Quantum series" even though the stored name is just "Quantum series".

A family whose members all share one dashboard category (Yamaha's two Rio-D2 racks are both "I/O Racks") is filed under it normally, same as any other row. One spanning several categories (Yamaha's Rivage

PM components live under Consoles, DSP Engines, and I/O Racks) isn't attributed to any one of them arbitrarily - it instead gets its own dedicated subsection, named in `FAMILY_CATEGORIES` (also in `app/families.py`), e.g. "Rivage PM Series". That's only set for families where it's actually useful - one that's the only thing tracked under its manufacturer (Allen & Heath's dLive series) stays without a heading instead, since a heading identical to the one row underneath it would just be noise. A manufacturer's subsections don't have to sort alphabetically or by equipment type either: Yamaha's "Rivage PM Series" is pinned ahead of "Consoles" in `CATEGORY_ORDER` (`app/dashboard_data.py`) since it's their flagship line, not because any general ordering rule would put it first.

DiGiCo's SD10 is a genuine singleton (no other SD-series console is tracked), but it still renders as a family-style row - "SD series", with a chevron expanding to a lone "SD10" pill - rather than falling back to a plain, non-expandable row the way every other singleton does. That's `FAMILY_FORCE` in `app/families.py`: a checker_key can opt into the family treatment even with one current member, when (like DiGiCo SD) its FAMILY_NAMES entry represents a whole product line rather than describing one specific model. Its dashboard category is just "Consoles" though, same as the Quantum family right above it - no dedicated subsection, since both are genuinely the same equipment type.

A family's members are supposed to always report identically (same `CheckResult`, one `check_all()` call), but that premise could still be violated by something outside any single check run - a member added to the family before its first real check, stale state from before it joined, a future checker bug. So `_consolidate_families` doesn't just trust the first member for the row's displayed status: it picks whichever member's status is most urgent (error beats a pending update beats "ok"), so a failing member can't be quietly absorbed into a row that otherwise looks fine. A family with any failing member shows "Check failed" on its row. The error line underneath is the message once when the whole family failed together (the normal case - one shared fetch), and names specific models only when just some failed or the messages differ. There's deliberately no per-model marker in the expanded pills - with every current checker assigning one shared result per `checker_key`, a single member failing alone can't happen in practice, so the row-level status is all that's needed.

Rows within a manufacturer are alphabetical by default; `MODEL_ORDER` in `app/dashboard_data.py` overrides that where it helps. Shure's is a fixed order following its release lines - Shure's own release notes name "Axient Digital 1.6.18" and "Axient Digital PSM 1.3.6" as system versions - so rows that share a version sit together: AD4Q, ADX1/ADX2, AD610, SBC240, then ADTQ, ADXR, SBC441, then the rest. It's deliberately fixed rather than a sort by whatever version each row currently shows, so rows don't jump around if one product's version ever diverges from the rest of its line (it has before - AD610 shipped a 1.4.33 the others never got). Those products stay separate rows with separate checks and separate Slack messages: Shure publishes them as separate firmware listings that currently agree, not as one package.

The stats panel's two counts deliberately mean different things. "Tracked" is a coverage number - every real, individually-tracked piece of equipment, unaffected by how many rows that collapses into on screen, so it doesn't shrink just because some of what it covers shares one firmware source. "Updates" counts rows instead, not units - if Rivage PM's 8 components all pick up a new version at once, that's genuinely one thing to go look at, so it shows "1 update available," not 8. Clicking the Updates card filters straight to those rows, so the number it shows always matches what you'd actually see land on screen. "Errors" still counts

units (unchanged) - see `build_dashboard_context` in `app/dashboard_data.py` for exactly which list each one sums over.

Deployment

The published site (on Vercel) is a **static build**, not the live FastAPI app — there's no server running continuously anywhere. Instead:

- `.github/workflows/daily-check.yml` runs once a day (GitHub Actions cron, plus a manual "Run workflow" button). It rebuilds a throwaway database from `data/equipment.json`, runs every checker for real, writes the results back to `data/equipment.json`, renders the dashboard to `public/index.html` (`scripts/build_static.py`), and commits both back to the repo — a fresh "Last checked" timestamp lands every single day, even when no version actually changed, so there's always a commit to show the check ran.
- **Vercel** serves the `public/` directory as-is (see `vercel.json`) — no build step, no server, just static files. Importing this repo into Vercel needs no extra configuration.
- The dashboard has no interactive controls that write anything (no manual "Log" or "Acknowledge" buttons) — it's a pure status *display*, matching what a backend-less static site can actually support. Update flags clear themselves automatically instead (see below).
- The site asks search engines not to index it, via a `<meta name="robots" content="noindex, nofollow, noarchive">` tag in the template. That's a request well-behaved crawlers honor, not access control - anyone with the link can still open it, and it doesn't cover the public GitHub repo itself (GitHub has no setting to keep a public repo out of search). An `X-Robots-Tag` header via `vercel.json` was tried too, but Vercel never sent it (cause not found), so it was removed rather than left as config that doesn't do anything. There's deliberately no `robots.txt` blocking crawling: a crawler has to be allowed to fetch the page to see the noindex instruction.
- `data/tracker.db` (SQLite) is still what the local interactive app uses day-to-day and is gitignored, same as always - it's not part of the deployment at all.

Slack notifications

The daily workflow posts to Slack (`app/slack.py`) whenever it finds a **genuinely new** firmware version - comparing each item's version before and after that day's run, not just whatever's currently flagged. A version that's still pending from a previous day, or one that just auto-cleared after its 1-month window, doesn't trigger a repeat message; only an actual change does.

Items are grouped by family before posting - e.g. all 7 DiGiCo Quantum consoles updating together to the same version becomes one message ("New firmware available for *DiGiCo Quantum series* → V23"), not seven. Grouped by `checker_key` (items that share one get checked together and always report the same new version, by construction - a more reliable "these are really the same family" signal than just matching version-number strings, which could coincidentally collide between two unrelated manufacturers), using the same `FAMILY_NAMES` mapping in `app/families.py` that the dashboard uses to consolidate rows - see Firmware families - so the two never disagree about what counts as one family. A genuinely unrelated update

detected the same day (different `checker_key`) always gets its own separate message, `@channel` and all - never bundled into someone else's.

Retries before anything counts as failed. A failed check is retried up to twice more, 30 seconds apart (`RETRY_ATTEMPTS` / `RETRY_DELAY_SECONDS` in `app/runner.py`), and only the items that failed are re-sent to their checker. Only the final outcome is recorded, so a momentary problem - a bot-protection page, a half-rendered page - that clears on retry never shows "Check failed" and never reaches Slack. This was added after the Allen & Heath dLive check failed once with "Version pattern not found on page" and was fine the next day. A checker that really is broken still fails after the retries (costing about a minute extra per run), and the retries are logged in the Actions run.

Check-failure alerts (a separate channel). When an item's check *newly* fails, a message goes out ("⚠️ Firmware check failed for *Yamaha Rivage PM series*: ") - and a matching "✅ ... recovered (now V7.20)" when it comes back, so a failure alert is never left hanging. Both go to the **alerts** channel via `SLACK_ALERTS_WEBHOOK_URL`; firmware-update alerts keep going to the main channel via `SLACK_WEBHOOK_URL`. If the alerts secret is unset, failures and recoveries fall back to the main webhook rather than disappearing. Details:

- **Once per incident.** `find_new_errors` / `find_recoveries` in `app/changes.py` compare each item's status before and after the run, so a failure alerts when it starts and "recovers" when it ends - not every morning in between. A failure that clears on a retry within one run never alerts and so never "recovers".
- **Families.** Members that share one fetch are one message, with the error stated once.
- **Who's pinged.** Failures ping one person (`SLACK_ALERT_USER_ID`), not `@channel`. Recoveries ping no one.
- **Why it's needed.** A single broken scraper doesn't fail the workflow (the runner records it and carries on), so the workflow-level alert never fires for it. That alert - "daily run failed", `scripts/notify_run_failed.sh` - also goes to the alerts channel.
- **What the failure says.** When a page loads but the version isn't on it, the message now includes what the page actually was - `(HTTP 200, title "...", starts "...")`, built by `describe_page` in `app/checkers/base.py` - so a bot-protection page or redesign is recognisable from Slack alone. This covers the Allen & Heath, Dante, d&b, Shure and Yamaha checkers; DiGiCo and SSL read a JSON API, so there's no page to describe. That page text comes from a third-party site, so it's capped, escaped for Slack (a page can't smuggle in an `@channel`), and the static dashboard build has HTML autoescaping on for the same reason.

Not covered: a failed Vercel deploy.

Reads the webhook URL from the `SLACK_WEBHOOK_URL` repository secret (GitHub → Settings → Secrets and variables → Actions), passed to the workflow step as an env var - never hardcoded, and not logged anywhere (GitHub also masks any exact occurrence of a registered secret's value in the Action's log output automatically). Missing the secret, or the POST itself failing, never breaks the daily commit - `notify_updates()` in `app/slack.py` degrades to a no-op (logged, not raised) in either case.

Testing Slack (new webhook, new recipient). GitHub -> Actions -> "Daily firmware check" -> Run workflow -> tick `slack_test`. That skips the firmware checks and the commit and posts, with the real secrets, clearly-labelled TEST messages (never `@channel`):

- to the **updates** channel (`SLACK_WEBHOOK_URL`): a connection check;
- to the **alerts** channel (`SLACK_ALERTS_WEBHOOK_URL`): a sample *failure* alert and a sample *recovered* message about a fake "TEST Sample item", built by the same code real alerts use, so they look exactly like the real thing - that's how you trigger a "test failure" without breaking anything;
- also to the alerts channel: a sample "daily run failed" alert, through the same `scripts/notify_run_failed.sh` the real failure path uses.

Each mentions the alert recipient (`SLACK_ALERT_USER_ID`) so you can see who real failures will ping. The run goes red if Slack rejects any of it, with Slack's reason in the log (e.g. `no_service` = webhook revoked, `channel_not_found`), and the log says if `SLACK_ALERTS_WEBHOOK_URL` is unset (failures then use the main channel). Locally: `SLACK_WEBHOOK_URL=... SLACK_ALERTS_WEBHOOK_URL=... SLACK_ALERT_USER_ID=U01ABCDE2F python3 scripts/slack_test.py` . A webhook posts to the one channel picked when it was created, and member IDs belong to a workspace - so moving to a different workspace means replacing *all three* secrets.

Secrets

Three GitHub Actions secrets (repo Settings -> Secrets and variables -> Actions), all for Slack. Their values are never in the repo and GitHub won't show them again once saved - to change one, save a new value over it. Vercel needs none, and `GITHUB_TOKEN` is supplied by GitHub automatically.

Secret	What it is	What uses it	If it's unset
`SLACK_WEBHOOK_URL`	Incoming-webhook URL for the main channel (#firmware-tracker)	Firmware-update alerts (`@channel`)	No update alerts are sent (the run still succeeds); failures with no alerts secret also lose their fallback
`SLACK_ALERTS_WEBHOOK_URL`	Incoming-webhook URL for the private alerts channel	Check-failure alerts, "recovered" messages, and the "daily run failed" alert	Those fall back to `SLACK_WEBHOOK_URL` , so they still arrive, just in the main channel
`SLACK_ALERT_USER_ID`	Slack member ID (like `U01ABCDE2F`) of the one person pinged on failures	The mention on failure alerts, the "daily run failed" alert, and test messages	The same messages post with no mention

A webhook always posts to the one channel picked when it was created, so sending something elsewhere means a new webhook and a new secret value. Member IDs belong to a workspace. Whether the secrets are working is checked by the manual `slack_test` run (see above), not by reading them.

Local automatic checking

If you run this locally instead of relying on the deployed site, the background scheduler (`app/scheduler.py`, APScheduler) only runs **while the app's process is alive** — if it's just open on your laptop, it only checks while your laptop has it open. The deployed site doesn't have this limitation, since GitHub Actions runs the check regardless of whether anyone's computer is on.

When a version changes, the item gets an "Update available" badge and an amber row highlight, in its normal position - it isn't moved to the top. Both clear themselves automatically 1 month after the change was *first* detected (`Equipment.last_changed_at`, see `app/runner.py`'s `UPDATE_HIGHLIGHT_WINDOW`) - re-confirming the same pending version on later daily checks doesn't restart that clock, only a genuinely new version change does. No manual acknowledgment involved anywhere.

Project layout

```
app/
  main.py            FastAPI routes + dashboard
  dashboard_data.py  Grouping/sorting/summary logic shared by the live app
                     and the static-site builder, incl. family row
                     consolidation (see Firmware families)
  families.py        FAMILY_NAMES: checker_key -> brand-free family name,
                     shared by dashboard_data.py and slack.py (the latter
                     prepends the manufacturer back on)
  models.py          Equipment / CheckLog tables (SQLAlchemy)
  runner.py          Runs checkers, diffs versions, writes history,
                     auto-expires update flags after UPDATE_HIGHLIGHT_WINDOW
  scheduler.py       Background interval job (local interactive use only)
  seed.py            The 48-item equipment list + which checker covers each
  export.py          Equipment <-> data/equipment.json round-trip, used by
                     the CI pipeline
  changes.py         Before/after diff of two JSON snapshots -> the genuine
                     version changes Slack is told about
  slack.py           Webhook notifications, one message per family
  checkers/
    digico.py, yamaha.py, ssl.py                      Plain-HTTP (HTML / Zendesk JSON API)
    allenheath.py, dante.py, dbaudio.py, shure.py     Headless-browser (Playwright)
    browser_base.py    Shared Playwright helper
    registry.py         Routes a checker_key to its module
  templates/, static/  Dashboard UI
scripts/
  ci_check.py        Entry point for the daily GitHub Actions run
  build_static.py    Renders the dashboard to public/ for Vercel
  make_pdf.py        README.md / TODO.md -> the committed README.pdf / TODO.pdf
data/tracker.db      SQLite database (created on first run, local dev only)
data/equipment.json  Git-committed source of truth for the deployed site
```

Adding equipment

Add a row to the `EQUIPMENT` list in `app/seed.py` (manufacturer, model, category, checker_key, source URL, optional notes), then restart the app (or run `python3 -m app.seed`) — `seed()` fully syncs the database to match the list: new rows are added, changed rows are updated, and rows removed from the list are deleted.